

PDFEngine

Capability Proof Sheet

Every page of this document is a test. Each section names a real defect or a real gap in the browser engine underneath, then demonstrates the fix in the same place you are reading about it — so the claim and the evidence are never separated.

Nothing here is illustrated with a screenshot. This PDF was produced by the engine it describes.

Covers

Release blockers RB-1 to RB-6 · Page-level typesetting T1-1 to T1-9

Generated by PDFEngine · A4 · two-pass render with real page resolution

Contents

The page numbers below are not typed in. They are read back out of the finished PDF and substituted on a second pass.

Running headers and footers (T1-1)	3
Cross-references and leaders (T1-2, T1-3)	4
Cover and mirrored margins (T1-4)	5
Footnotes (T1-5)	6
Widow and orphan control (T1-6)	7
Named pages (T1-7)	8
Page floats (T1-8)	9
Per-page reservation (T1-9)	10
What the file can be USED for (Tier 2)	11
Attachments and e-invoicing (T2-1)	12
Digital signatures (T2-2)	13
Fillable form fields (T2-3)	14
Page operations (T2-4)	15
Fast web view (T2-5)	15
Print production (T2-6)	16
Release blockers RB-1 to RB-6	17
What this engine does not do yet	17

Running headers and footers

T1-1 · STRING-SET + @PAGE MARGIN BOXES

THE ISSUE

Every real report needs the current chapter in its header. Chromium's PDF export takes `headerTemplate` — *one* fixed template for the entire document. It cannot change per page, so "the current chapter" is impossible through it. Chromium also discards `string-set` before the stylesheet is ever exposed to script, so the value cannot even be read.

THE FIX

The engine reads `string-set` from the raw stylesheet, records every assignment, then resolves which page each one landed on *by searching the finished PDF* — not by estimating from the DOM, which was measured wrong twice. The header is then drawn per page, the same mechanism that stamps watermarks.

SEE IT HERE

Top-left of this page reads

RUNNING HEADERS AND FOOTERS

. Turn the page and it changes by itself. Bottom-right counts real pages, and the total is the real total.

The value carries forward correctly too: a section long enough to span several pages keeps its title in the header on the pages that contain no heading at all.

Cross-references and leaders

T1-2 · `TARGET-COUNTER()` T1-3 · `LEADER()`

THE ISSUE

A page number does not exist until the document has been paginated. Two earlier attempts computed them from DOM geometry and *both* produced wrong numbers: counting forced breaks missed pages created by ordinary overflow, and dividing by scroll height mis-attributed pages because forced breaks make DOM coordinates non-linear. Separately, the engine only understood its own proprietary attribute, so migrating a Prince or DocRaptor template meant rewriting it.

THE FIX

Render once, locate each target's text in the *real* pages, substitute the true numbers, render again — the same multi-pass approach Prince itself uses. Standard `target-counter(attr(href), page)` is translated onto that resolver, so existing templates work unmodified. `Leader(' . ')` fills the space between the title and the number.

SEE IT HERE

Go back to
[CONTENTS](#)

. The dots are the leaders. The numbers are real — check any one of them against the page it points to.

A reference that cannot be resolved renders as `?` plus a diagnostic, never as a silent blank — a reader cannot tell an empty gap from a design choice.

Cover pages and mirrored margins

T1-4 · @PAGE :FIRST / :LEFT / :RIGHT

THE ISSUE

These looked broken and were logged as missing engine work. Measuring showed Chromium *already implements all three*. They were dead for a different reason: the HTML sanitizer necessarily drops CSS it does not recognise, and it was stripping every `@page` descriptor on the way through — including `size`, which is why every document silently came out A4 regardless of what it asked for.

THE FIX

Stop stripping `@page`, and parse paged-media CSS from the pre-sanitizer source. This one needed a gate, not an implementation — and it is the reason the sanitizer is now the first suspect whenever a paged-media feature does nothing.

SEE IT HERE

Page 1. Its margins are visibly wider than every other page in this document. That is `@page :first { margin: 52mm 30mm }` doing exactly what it says.

Footnotes

T1-5 · FLOAT: FOOTNOTE

THE ISSUE

Legal, academic and financial documents need footnotes. Chromium supports none of the mechanism: content marked `float: footnote` renders *inline, exactly where it was written*¹, and neither `::footnote-call` nor `::footnote-marker` numbers anything.

THE FIX

The engine lifts each footnote out of the flow and leaves a numbered call marker behind. It then resolves which page that call landed on from the finished PDF, reserves space at the bottom of the page, and re-renders — because adding footnote area changes pagination and can move a call to a different page².

SEE IT HERE

Two superscript markers above, and the matching notes at the foot of this page, below the rule — not where they were written in the source.

Both notes were authored in the middle of the paragraphs above. Neither appears there.

1. Measured 2026-08-18: the test marker landed 16% of the way down page 1 instead of at the page bottom.

2. The loop iterates until every page holding a call has room, and reports rather than overlapping text if it cannot settle.

Widow and orphan control

T1-6 · ORPHANS / WIDOWS

THE ISSUE

Logged as needing a line-breaking engine. Measuring showed Chromium *implements orphans and widows correctly* — at a setting of 4, a paragraph that would have split 5/3 moved whole to the next page. The real gaps were smaller and more specific: the values were hard-coded to 2 with no way for a caller to change them, and the one case CSS *cannot* satisfy — a block taller than a page — degraded in silence.

THE FIX

Expose `orphans` and `widows` as options rather than re-implementing line breaking, and measure real line boxes so a block that can never satisfy the request is *reported* instead of quietly split.

SEE IT HERE

Nowhere in this document is a single line of a paragraph stranded by itself at a page boundary. That is the feature working; its success looks like nothing being wrong.

Named pages

T1-7 · @PAGE WIDE + PAGE: WIDE

THE ISSUE

A landscape fold-out inside a portrait report, or a cover with its own paper. CSS expresses it as `@page wide { size: A4 landscape } plus page: wide`. Chromium *silently ignores* it — measured, a section declared A4 landscape with 50mm margins came out with byte-identical geometry to the body pages. It cannot be corrected after rendering, because page size changes layout, not just what is stamped on top.

THE FIX

Consecutive content sharing a page name forms a *run*. Each run is rendered on its own paper and the parts are stitched into one document. Everything document-wide is then resolved against the stitched result — page counters, cross-references, footnote placement and the bookmark outline — because a run only knows about itself.

SEE IT HERE

This page is landscape. The pages before and after it are portrait, in the same file, with the same header and a continuous page count.

FEATURE	CHROMIUM ALONE	WITH PDFENGINE	HOW IT IS PROVEN
Running header from content	One fixed template for the whole file	Per page, from the real PDF	Differential render
target-counter()	Not implemented	Real page numbers	Checked against actual pages
Footnotes	Renders inline where written	At the foot of the right page	Coordinate assertion
Named pages	Silently ignored	Per-run paper, stitched	Page geometry assertion
Page floats	Renders inline where written	Pinned to the page edge	Coordinate assertion

Page floats

T1-8 · FLOAT: TOP / FLOAT: BOTTOM

THE ISSUE

A figure that lands halfway through a paragraph interrupts the reading. Print typesetting pins it to the top or bottom of the page instead. Chromium implements neither edge — measured, a `float: top` box and a `float: bottom` box landed at the *identical* position, indistinguishable from no float at all.

THE FIX

The same machinery as footnotes: lift it out of the flow, resolve its page from the finished PDF, reserve the band, re-render, draw. One honest difference — a float is arbitrary content, so it cannot be rebuilt from a text description. It is captured as an image at twice its size while the browser still has it laid out.

SEE IT HERE

Figure 1, at the foot of this page, below this text. In the source it sits above this paragraph.



Figure 1 — This block was written in the middle of the text below. It is printed at the foot of the page because it asks for `float: bottom`.

Per-page reservation

T1-9 · FOOTNOTERESERVATIONMODE

THE ISSUE

Reserving footnote space by growing the page margin is correct but blunt: it is uniform, so a document with one crowded page loses that band on *every* page, including the ones carrying no footnotes at all. Reserving per page instead costs one extra render per page — a page break moves every later page, so no two pages can be measured from the same render.

THE FIX

Neither strategy is right for every document, and the caller cannot tell which they need by looking at their HTML — it depends on how unevenly the footnotes fall, which only exists once the document has been paginated. So the engine measures both costs after the first render and

CHOOSES

. Measured: a document with five footnotes crowded onto one page reclaimed 76 points of height on a footnote-free page.

SEE IT HERE

The render diagnostics for this document contain a line beginning `Footnote reservation:` naming the strategy chosen, what the alternative would have cost in points, and how many extra renders it was worth. Set `footnoteReservationMode` to `"uniform"` or `"per-page"` to overrule it.

Why it is decided rather than configured: a switch left to the caller is a switch set wrong. Off by default and some documents silently lose a page of content; on by default and every document pays for renders it does not need.

What the file can be USED for

TIER 2 · PDF OUTPUT FEATURES

Everything up to this point changed how the document is LAID OUT. The sections that follow change what the finished file can be *used for* — and each one maps to a customer who could not be served without it.

This half is verified differently, and deliberately so. A typesetting claim can be checked by looking at the page. A claim that this is a valid e-invoice, or a signature a court would accept, cannot — so every one below is checked with an independent tool: veraPDF, poppler, qpdf or openssl. Never by reading back the engine's own output.

FEATURE	WHO COULD NOT BE SERVED WITHOUT IT	CHECKED WITH
Attachments (T2-1)	EU e-invoicing — Factur-X, ZUGFeRD	veraPDF, poppler
Signatures (T2-2)	Contracts, approvals, anything auditable	openssl
Form fields (T2-3)	Applications, consent, returns	Independent PDF reader
Page operations (T2-4)	Document assembly	Independent PDF reader
Fast web view (T2-5)	Large documents opened in a browser	qpdf
Bleed and crop marks (T2-6)	Commercial print	Page-box measurement

Attachments and e-invoicing

T2-1 · EMBEDDED FILES

THE ISSUE

An EU e-invoice is one PDF that a human reads and a machine reads: the page for the person, an embedded XML file for the buyer's accounting system. Without embedded files the engine cannot produce Factur-X or ZUGFeRD at all — not badly, at all — and an entire revenue vertical is closed.

THE FIX

Each file is written as an `/EmbeddedFile` stream plus a `/Filespec`, registered BOTH in the catalog's name tree (what the attachment pane lists) and in `/AF` (what makes it an *associated* file, which PDF/A-3 requires). The backlog assumed a library change was needed; measuring showed the current library already exposed everything required.

SEE IT HERE

Open this document's attachment pane.

FACTUR-X.XML

is in it, and it extracts byte-identical to what was supplied. A PDF/A-2b request with an attachment is REFUSED with a 400, because PDF/A-2 permits only embedded PDF/A documents — producing it anyway would hand you an invoice your recipient rejects.

One bug worth keeping. A MIME type contains a `/`, which a PDF name cannot. Hand-escaping it to `text#2Fxml` got escaped again on save to `text#232Fxml`, and veraPDF failed PDF/A-3 clause 6.8. For an e-invoice that is a rejected invoice, not a cosmetic defect.

Digital signatures

T2-2 · PAdES B-T AND B-LT

THE ISSUE

A signature that merely exists is worthless. Three things have to be true: it must verify in tooling nobody here wrote; tampering must break it; and it must still verify after the signing certificate expires — which a bare signature does not, because the only evidence of *when* it was signed is a clock the signer controls.

THE FIX

A detached CMS signature over exactly the byte ranges the finished file declares, with the attributes CAdES requires. `timestampUrl` attaches an RFC 3161 timestamp from a trusted authority (PAdES B-T).

`embedValidationData` appends the certificate chain and its revocation lists as an incremental update, so the document carries its own evidence and validates with no network at all (PAdES B-LT).

SEE IT HERE

Open the signature panel. This document is signed, timestamped and carries embedded validation data. Change one byte of it and the signature fails.

What measurement forced. The PDF library's own signing could not be used: it hands the signer 2,133 bytes beginning with whitespace while writing a byte range declaring 2,922 beginning at the file header. `openssl` rejects the result. The signature is therefore computed here, over exactly what the finished file declares.

Not claimed: the signature is invisible (it seals the document and draws nothing), and signing cannot be combined with encryption or fast web view — both rewrite the bytes a signature seals, so both are refused rather than silently producing a signature that does not verify.

Fillable form fields

T2-3 · ACROFORM

THE ISSUE

Applications, consent forms and tax returns need fields a person can type into. This was recorded as

BLOCKED

on the PDF library for good reason: the field classes in both candidate libraries have zero public constructors, so there is no API to call.

THE FIX

The observation was true and the conclusion was wrong. A form field is not a library type — it is a widget annotation on a page plus an entry in the catalog's `/AcroForm`, and both are ordinary dictionaries this engine already writes by hand for attachments. What was missing was a convenience API, not the capability.

SEE IT HERE

The three fields below are real and fillable — click one and type into it. They are drawn by an appearance stream carried in the file, not by a request that the reader draw them, because several viewers ignore that request and show an invisible form.

Full name	
Email	
I agree to the terms	

How to check this without trusting the page. Run `python3 tests/verify_tier2.py`. It fills a field with an unrelated library, saves, reopens and reads the value back — along with fourteen other Tier 2 claims, each checked by poppler, openssl, qpdf or pypdf rather than by this engine.

The lesson, recorded so it is not repeated: “the library has no API for it” is not the same as “the format cannot do it”, and only the second is a blocker.

Page operations and fast web view

T2-4 · EXTRACT / ROTATE / FLATTEN / N-UP T2-5 · LINEARIZATION

THE ISSUE

Document assembly needs the mechanical operations: pull pages out, turn them, put several on a sheet, remove interactivity. Separately, a large PDF opened over HTTP makes the reader wait for the whole file before it can paint page one.

THE FIX

One `transform` endpoint for the page operations. `extract` preserves the order written, so `3,1` reorders and `1,1` duplicates — sorting the selection would quietly refuse what was asked. `rotate` is additive, so a page already turned is turned BY the amount asked rather than reset TO it. `nup` keeps text as text. `flatten` removes interactivity without rasterising, so the document stays searchable.

SEE IT HERE

Not visible on a page — these operate on a finished PDF. Post this document to `/api/v1/pdf/transform` with `operation: "nup"` and you get these pages two to a sheet, still selectable. Linearization is verified by asking qpdf, the reference implementation, whether the result is linearized.

Refused rather than silently dropped: fast web view plus signing. Applying the signature rewrites the document and undoes the page ordering, so you would get a file that is signed and not linearized, with nothing to tell you.

Print production

T2-6 · BLEED, CROP MARKS, PAGE BOXES

THE ISSUE

A commercial printer trims a sheet with a blade that lands a fraction out. If artwork stops exactly at the finished edge, the trim leaves a white sliver. The artwork has to run PAST the cut line, and the file has to record where that line is.

THE FIX

Bleed is applied at RENDER time — the page is rendered larger so backgrounds genuinely extend past the cut — and the finished size is recorded as the `TrimBox`, taken from the nominal paper rather than the rendered box so it is exact to a hundredth of a millimetre. Crop marks key off that rectangle and sit outside the bleed, so none prints inside the finished page.

SEE IT HERE

Not on this page, deliberately — a proof sheet you read on screen should not be trimmed. Render any template with `bleedMm: 3` and inspect its page boxes.

The bug this had, and how it was found. It was asked whether the feature was real or present.

Landscape was ignored: an A4 landscape job produced a PORTRAIT trim box, so every landscape print would have been cut wrong. It had only ever been tried on A4 portrait.

Not claimed: CMYK and PDF/X. Converting to CMYK needs a colour-management pipeline, and the usual engine for it is AGPL-licensed — a licensing decision, not an engineering one. No validator available here checks PDF/X, so conformance could not be proven even if attempted. Every print job says so at render time.

Release blockers RB-1 to RB-6

Six issues that each blocked release on their own. Two are visible in this document; four are infrastructure and are proven by gates, not by looking at a page. Both kinds are listed, and which is which is stated plainly.

RB-2 · ARABIC TEXT WAS UNUSABLE — VISIBLE HERE

Right-to-left text is stored in the PDF in *visual* order. Copying it out produced reversed, unusable text, which also meant screen readers could not read it.

FIXED

Logical-order `/ActualText` is attached to every reversed run. Not an option, because a text layer that cannot be copied is a defect rather than a preference.

هذه فقرة عربية يمكن نسخها بشكل صحيح

VERIFY IT

Select the Arabic line above and paste it into a text editor. It arrives in reading order, not reversed.

ID	WHAT WAS WRONG	HOW IT WAS CLOSED	VISIBLE HERE?
RB-1	Encryption was RC4-128 only — obsolete, and rejected by security review	AES-256, fail-closed: it never silently downgrades	No — set a password to see it
RB-2	Arabic text layer unusable when copied	<code>/ActualText</code> on RTL runs; extraction failures 1 → 0	Yes — above
RB-3	No automated check that text could be extracted at all	A 19-fixture extraction gate with a committed baseline	No — it is a CI gate
RB-4	Accessibility (PDF/UA) claimed but never validated	veraPDF PDF/UA-1: 106 checks, 0 failures	Partly — tagging is in the file
RB-5	Platform layer untested	A 17-check gate — which found a live cross-tenant data leak	No — it is a CI gate
RB-6	Colour profile redistribution licence unconfirmed	Resolved	No — it is a licensing matter

What this engine does not do yet

Kept deliberately short and specific. Every limitation this page listed in earlier revisions has now been closed. What follows is the record of what each one was and how it was answered — and one item that was never a gap, only a decision the engine now makes for you.

ITEM	WHERE IT STANDS	WARNED AT RENDER?
Links inside footnotes	Closed. A citation link in a footnote is drawn in link colour and carries a real PDF annotation, so it is clickable. The footnote's text arrives complete, with punctuation attached exactly as authored.	n/a — it works
Floated figures had no text layer	Closed. The picture is still what you see, but every word inside it is re-drawn over the image as invisible text at the same coordinates — the technique a scanned document uses for its OCR layer. A floated table is now selectable, searchable and readable by a screen reader.	n/a — it works
Bold and italic inside footnotes	Closed. Emphasis is drawn in a real bold or italic face. It needed two things: font files carrying those faces, and a font resolver — PdfSharpCore had none registered, so every family and every style resolved to one identical face, which also meant a font-family asked for in a margin box had never had any effect. Carlito, Caladea and Liberation Mono are bundled for it, all under the SIL Open Font License. Faking it by double-striking was built first and rejected: it thickens the stem but puts the word in the text layer twice, so the footnote copied out as 'Smith Smith v.v. Jones'.	Only if a Regular-only family is requested
Footnote space reservation	Not a gap — the engine decides. Reserving uniformly is one cheap re-render but costs every page the tallest band; reserving per page reclaims that at one render per page. Which is right depends on how unevenly the footnotes fall, which is only knowable after a render — so the engine measures it and picks, then says which it chose and what it was worth.	Yes — states the decision

Why any of this is printed here. The rule this engine follows is that degradation is REPORTED, never discovered. Anything above that affects your document appears as a warning on the render that produced it, naming how many elements were affected — so you learn it from the response, not from a customer.